

模板骨架 Template/ihxnan.hpp

用法：开题时把本文件整体粘贴到提交文件最上方，在 solve() 内写题解。

- 默认单组用例 (t=1)，多组用例时在 solve() 开头读入 t 即可 (main 里 while(t--))。
- cin / cout 已重定向到快速读写 fin / fout；**仅支持整数、char、string** (double 等类型编译期报错，与提交模式一致，避免“本地能编、评测报错”)。
- 本地调试编译需 -std=c++20 -DIHXNAN (dbg.hpp 依赖 C++20 source_location)，提供 gdb() 变量打印、gdbif() 条件打印、gdb_assert() 断言、TIME() 计时；提交时不加该宏，仅 gdb() / TIME() 退化为空操作——
gdbif() / gdb_assert() 在提交版会编译错误，需删除或包进 #ifndef IHXNAN。
- 快写基于缓冲区，程序结束由析构统一 flush，勿在中间混用 printf。
- rd0(x) 一行整体读入容器 (vector / array / deque 等，不含 map 嵌套)，rd1(x) 从 x[1] 读到末尾 (适配 1-indexed 容器)；分块宏 bl/bi/br 需先定义 blen (br 还需 n)，fid/gx/gy 需先定义 m。
- iINF / mod / IINF / MOD 均为 const ll，**不可再赋值** (双模数等场景请自定义变量)。
- fin 读到 EOF 后 ok 置 false，可用 if(fin) 判断输入是否结束 (适合未给组数的多组输入)。

```
#include <bits/stdc++.h>

// ===== 常用宏 =====
#define endl '\n' // 输出换行 (配合下方 IO 结构体，末尾无多余空白)
#define ls(x) ((x) << 1) // 线段树/树状结构：节点 x 的左儿子下标 (x*2)
#define rs(x) ((x) << 1 | 1) // 线段树/树状结构：节点 x 的右儿子下标 (x*2+1)
#define bl(x) ((x) * blen + 1) // 分块：块 x 左边界下标 (1 起始，含；需已定义 blen)
#define bi(x) (((x) - 1) / blen) // 分块：下标 x 所属块编号 (0 起始；需已定义 blen)
#define br(x) (min(((x) + 1) * blen, n)) // 分块：块 x 右边界下标 (受全局 n 限制，含；需已定义 blen、n)
#define fid(x, y) ((m) * (x - 1) + (y)) // 二维坐标 (x,y) → 一维编号 (1-indexed；需已定义 m)
#define gx(id) (((id) - 1) / m + 1) // fid 逆变换：编号 → 行号 (需已定义 m)
#define gy(id) (((id) - 1) % m + 1) // fid 逆变换：编号 → 列号 (需已定义 m)
#define rd0(x) for (auto &p : x) cin >> p // 容器整体读入 (vector/array/deque 等，不含 map 嵌套)
#define rd1(x) for (int i = 1; i < x.size(); ++i) cin >> x[i] // 从 x[1] 读到末尾 (1-indexed 容器，跳过 x[0])

// ===== 主框架 =====
int t = 1; void solve();
int main() { while (t--) solve(); return 0; }

// ===== 类型别名 =====
using namespace std;
using ll = long long;
using ul = unsigned long long;
using lll = __int128;
using ld = long double;
using vb = vector<bool>;
using vi = vector<int>;
using vvi = vector<vi>;
using vl = vector<ll>;
using vvl = vector<vl>;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using ti = tuple<int, int, int>;
using tl = tuple<ll, ll, ll>;

// ===== 常量 (const ll, 不可再赋值) =====
const ll iINF = 0x3f3f3f3f, mod = 998244353, lINF = 0x3f3f3f3f3f3f3f3f, MOD = 1000000007;

#ifndef IHXNAN
// 本地调试模式：dbg.hpp 提供同逻辑 IO 与调试宏 (需先 symlink；编译加 -std=c++20)
```

```

#include <dbg>
#else
// 提交模式: cin/cout 重定向到快读快写; 调试宏仅 gdb/TIME 有空操作回退
#define cin fin
#define cout fout
#define gdb(...) ((void)0)
#define TIME(name) ((void)0)

struct IO {
    static const int S = 1 << 21; // 输入/输出缓冲区大小
    uint8_t buf[S], *p1, *p2;
    int st[105], Top;           // st 为输出数位栈
    bool ok;                   // 读到 EOF 后置 false, 可用 if (fin) 判断
    IO() : p1(nullptr), p2(nullptr), Top(0), ok(true) { st[0] = 0; }
    ~IO() { clear(); }
    inline void clear() { fwrite(buf, 1, Top, stdout); Top = 0; }

    // 读一个字节 (缓冲区空则批量 fread)
    inline int inc() {
        if (p1 == p2) {
            p2 = (p1 = buf) + fread(buf, 1, S, stdin);
            if (p1 == p2) return EOF;
        }
        return *p1++;
    }
    // 写一个字节 (缓冲区满则落盘)
    inline void outc(int c) {
        if (Top == S) clear();
        buf[Top++] = (uint8_t)c;
    }
    inline explicit operator bool() const { return ok; }

    // 输出整数 (仅算术类型)
    template <typename T>
    inline IO &operator<<(T x) {
        int f = 0;
        if constexpr (std::is_signed_v<T>) { if (x < 0) outc('-'), f = 1; }
        do { st[++st[0]] = f ? -(x % 10) : x % 10, x /= 10; } while (x);
        while (st[0]) outc(48 ^ st[st[0]--]);
        return *this;
    }
    // 输出字符 / 字符串
    inline IO &operator<<(char c) { outc(c); return *this; }
    inline IO &operator<<(const char *x) { while (*x) outc(*x++); return *this; }
    inline IO &operator<<(char *x) { while (*x) outc(*x++); return *this; }
    inline IO &operator<<(const string &x) { for (auto c : x) outc(c); return *this; }

    // 读入整数 (跳过非数字, 支持负号; EOF 置 ok=false)
    template <typename T>
    inline IO &operator>>(T &x) {
        x = 0;
        int f = 0, ch = inc();
        while (ch != EOF && !isdigit(ch)) { if (ch == '-') f ^= 1; ch = inc(); }
        if (ch == EOF) { ok = false; return *this; }
        while (ch != EOF && isdigit(ch)) x = (x << 3) + (x << 1) + (ch ^ 48), ch = inc();
        f ? x = -x : 0;
        return *this;
    }
    // 读入单个字符 (跳过空白)
    inline IO &operator>>(char &x) {
        int c = inc();
        while (c != EOF && isspace(c)) c = inc();
        if (c == EOF) { ok = false; return *this; }
        x = (char)c;
    }
};

```

```

        return *this;
    }
    // 读入空白分隔的字符串
    inline IO &operator>>(string &x) {
        x.clear();
        int c = inc();
        while (c != EOF && isspace(c)) c = inc();
        if (c == EOF) { ok = false; return *this; }
        while (c != EOF && !isspace(c)) x += char(c), c = inc();
        return *this;
    }
} fin, fout;
#endif

```

Python 版骨架 Template/ihxnan.py

快速 IO 辅助函数：I() 读一行去空白、II() 读整数、MII() 读一行整数 map、LII() 读一行整数列表、GMI() / LGMI() 为 0-indexed 版（整体减一）。

```
import math
import sys
from collections import Counter, defaultdict, deque
from heapq import heapify, heappop, heappush

input = sys.stdin.readline
sys.setrecursionlimit(1000000)

def I():
    return input().strip()

def II():
    return int(input())

def MII():
    return map(int, input().split())

def LII():
    return list(map(int, input().split()))

def GMI():
    return map(lambda x: int(x) - 1, input().split())

def LGMI():
    return list(map(lambda x: int(x) - 1, input().split()))
```